Transportation Demand Forecast System

Predicting demand for shared vehicles by zone and time

Tom Bitran · ID 322373028

Lee Rosenblit · ID 322357880

The Problem

A persistent mismatch between the supply of shared vehicles and demand that shifts across space and time

What goes wrong today

- Long wait times for riders in some areas
- Empty vehicles circling with no demand
- Unnecessary traffic congestion
- Rising air pollution from wasted mileage

Why it matters

Better matching of vehicles to demand means shorter waits, less empty driving, lower congestion and emissions.

Our goal

Analyze historical trips and external signals to forecast demand per zone × time window.

The Dataset

NYC TLC Trip Record as the main dataset

263	100M+	2009	Parquet
Taxi Zones	Trips / Year	Records Since	Columnar Format

Primary source — NYC TLC

Content: Trip records: timestamps, pickup/dropoff zone, passengers, distance, fare, tip

Structure: Parquet columnar; Zone IDs (PU/DO LocationID) since 2016

Volume: Hundreds of millions of trips per year, from 2009 onward

Source: NYC.gov TLC Trip Record portal (+ Zone Shapefiles)

Supplementary sources

Open-Meteo

Free weather API - rain, temperature per zone & hour

NYC events

MSG, Broadway and holiday calendars

Taxi Zone Shapefiles

Zone polygon geometries for the geospatial join

Why Big Data?

Hundreds of millions of trips = not mission for a single machine

Volume

100M+ trips per year, since 2009

Variety

Trips joined with weather and events. 263 zones with polygon geometry.

Velocity

Constant streaming to compute demand in real time.

Distributed

Processed with PySpark on a cluster, parallel storage.

Existing Approaches - Pros & Cons

Three families of solutions for demand forecasting, and their trade-offs

Naive / historical average

PROS

- Simple, fast, interpretable
- Natural baseline to beat

CONS

- Ignores weather & events
- Fails on demand spikes

Classical ML & clustering (Our pick)

PROS

- Scales on Spark MLlib
- Captures space–time patterns

CONS

- Manual feature design
- Limited on complex non-linearities

Deep learning (CNN + LSTM)

PROS

- Highest accuracy
- Fuses space, time & semantics

CONS

- Compute-heavy & data-hungry
- Harder to scale / deploy

Proposed Solution

1

Data Storage

Load TLC trips + Open-Meteo weather + NYC events (Parquet/CSV)

→

2

Stream Processing

Clean, filter bad trips, aggregate by hour × zone × day

→

3

Geospatial Join

Apache Sedona — LocationID to zone polygons

→

4

Clustering

Spark MLlib K-Means — demand clusters by zone & hour

→

5

Analysis & Eval

Elbow (WCSS), visualization, MAE/RMSE vs. baseline

Our Novelty

A **multi-signal demand model** that fuses TLC trips with weather and events - capturing the real drivers of demand, not just typical averages.

real-time live prediction: the system generates on-demand forecasts from live input (zone, time, conditions), not only batch analysis of historical data.

Academic Background

How each paper relates, what we learned, and what we will use

Apache Sedona / GeoSpark

Yu, Zhang & Sarwat (2019) · GeoInformatica 23(1)

Learned: distributed spatial indexing (R-Tree/Quad-Tree) and joins. Will use: Sedona for the trip-to-zone geospatial join.

DMVST-Net for taxi demand

Yao, Wu, Ke et al. (2018) · AAAI

Learned: fusing spatial (CNN), temporal (LSTM) and semantic views. Will use: its space-time framing to design our features.

NYC Taxi trip-pattern analysis

Ferreira, Poco, Vo, Freire & Silva (2013) · IEEE TVCG (VAST) 19(12)

Learned: spatio-temporal exploration of this exact dataset. Will use: guidance for our clustering and visualization.

Open Questions & Theory

Open questions

- How many clusters (K) best capture demand?
- Do weather & events meaningfully beat trips-only?
- Is streaming worth it, or is batch enough?
- If a trip goes between several zones, where do you include it?

Theoretical background

- Distributed computing - Spark RDD
- Spatial indexing & joins
- Unsupervised clustering - K-Means, WCSS / elbow
- Evaluation - MAE / RMSE against baselines

Discussion

- Are weather and events worth the added complexity?
- Is a streaming path needed for this use case?
- Feedback on our proposed novelty?